



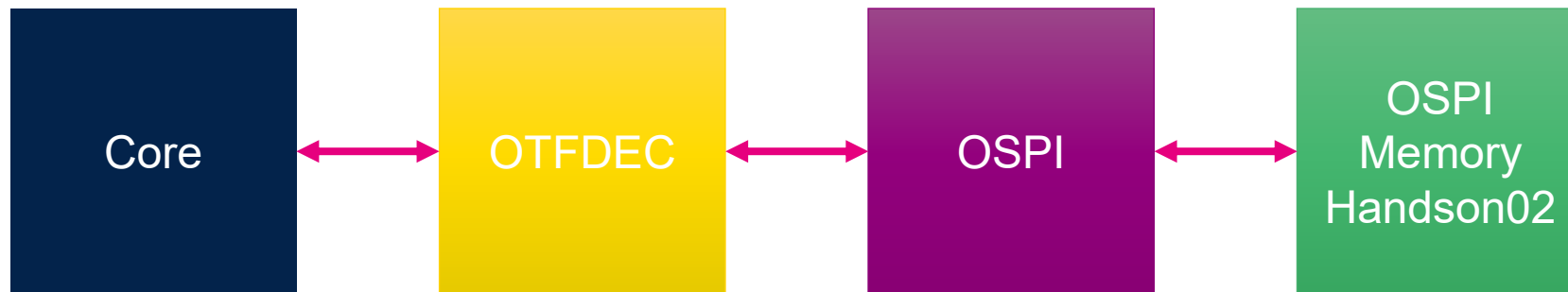
life.augmented

Code execution from external memory Hands On



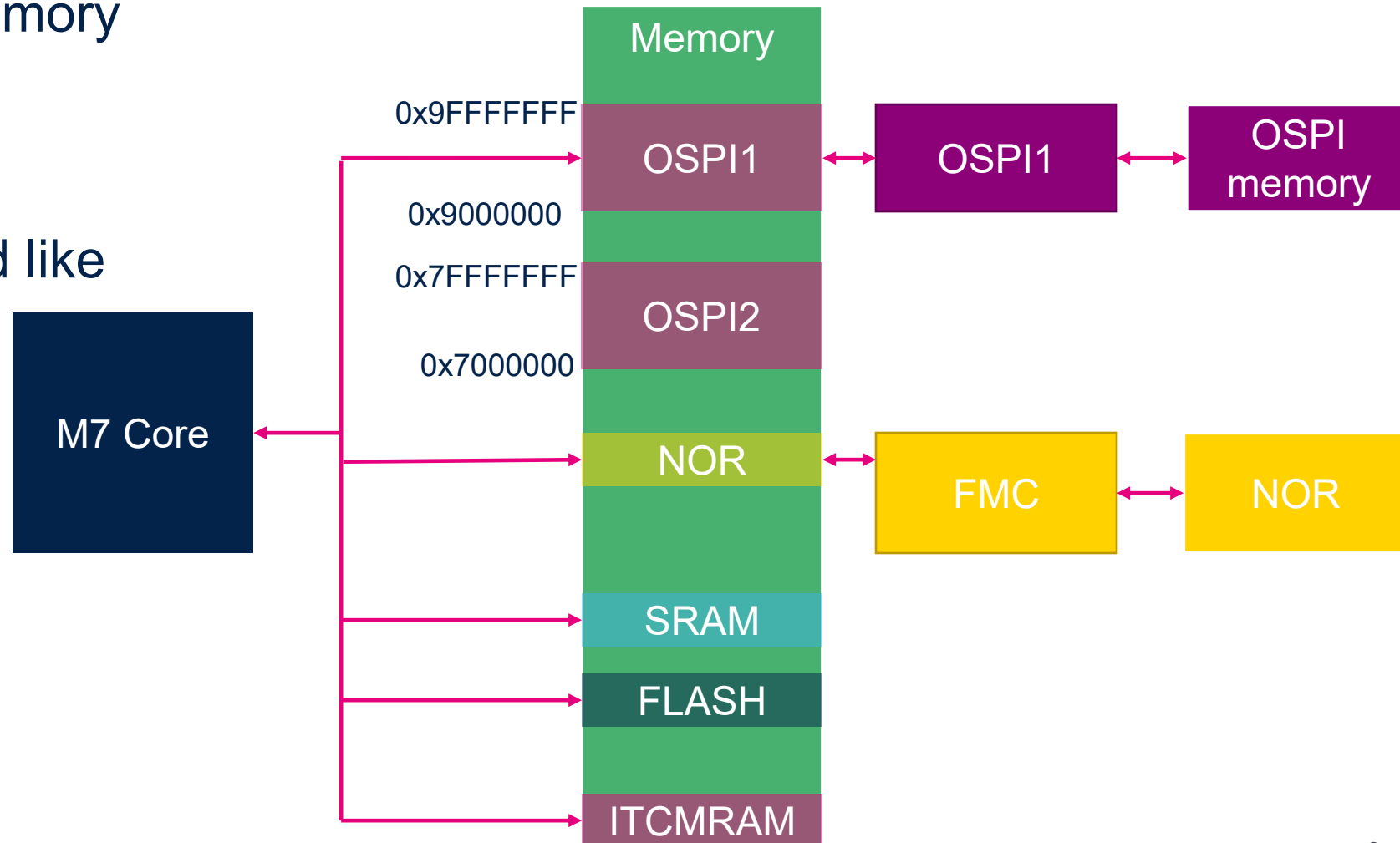
Target of this lab

- Move code from Handson02 external OSPI memory
- Encrypt the Handson02
- Use OTFDEC to decrypt this image



Executing code from external memory

- For devices with small memory
- External memory mapped to address space
- M7 can automatically read like internal memory
- We need first to enable external memory



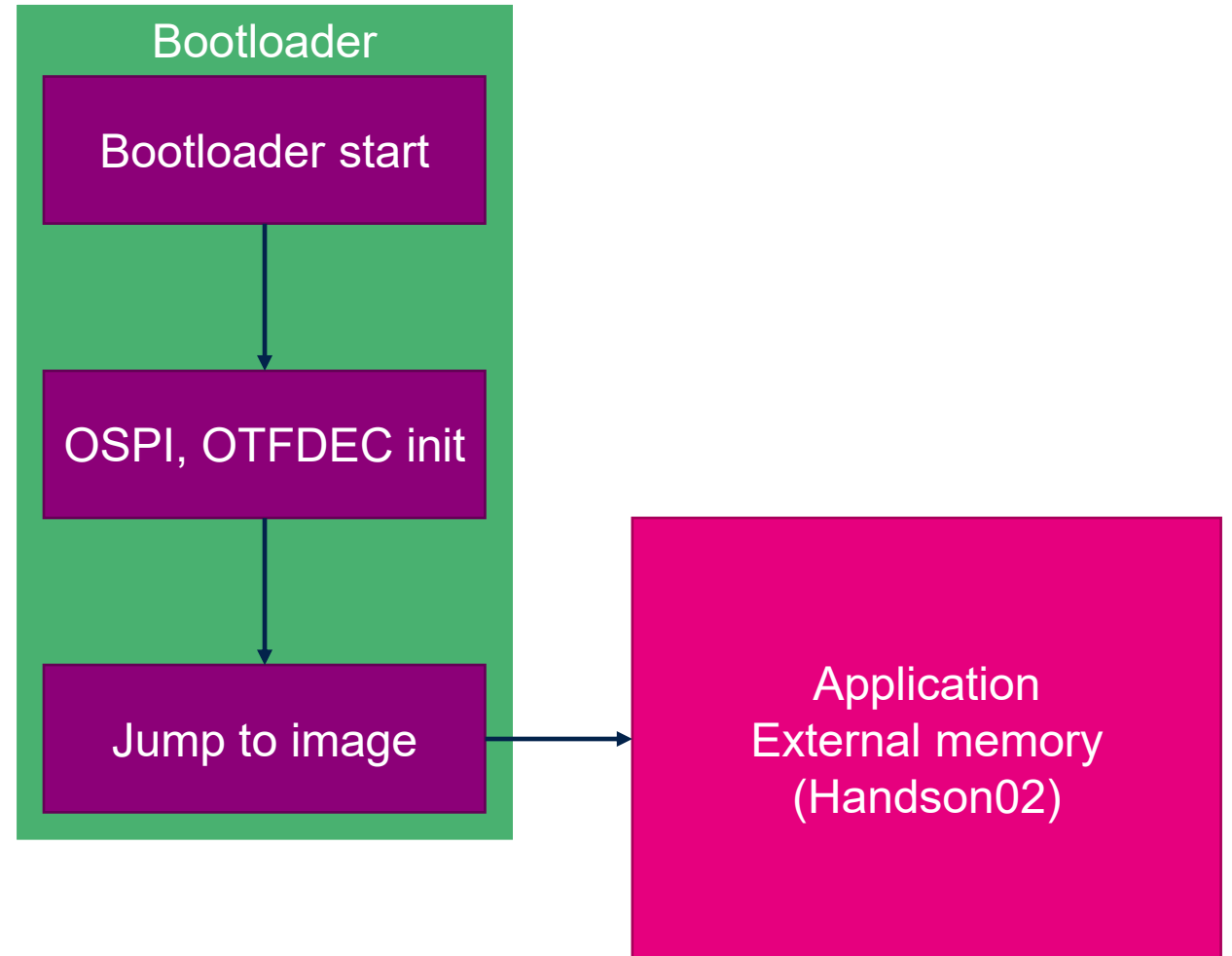
What is bootloader



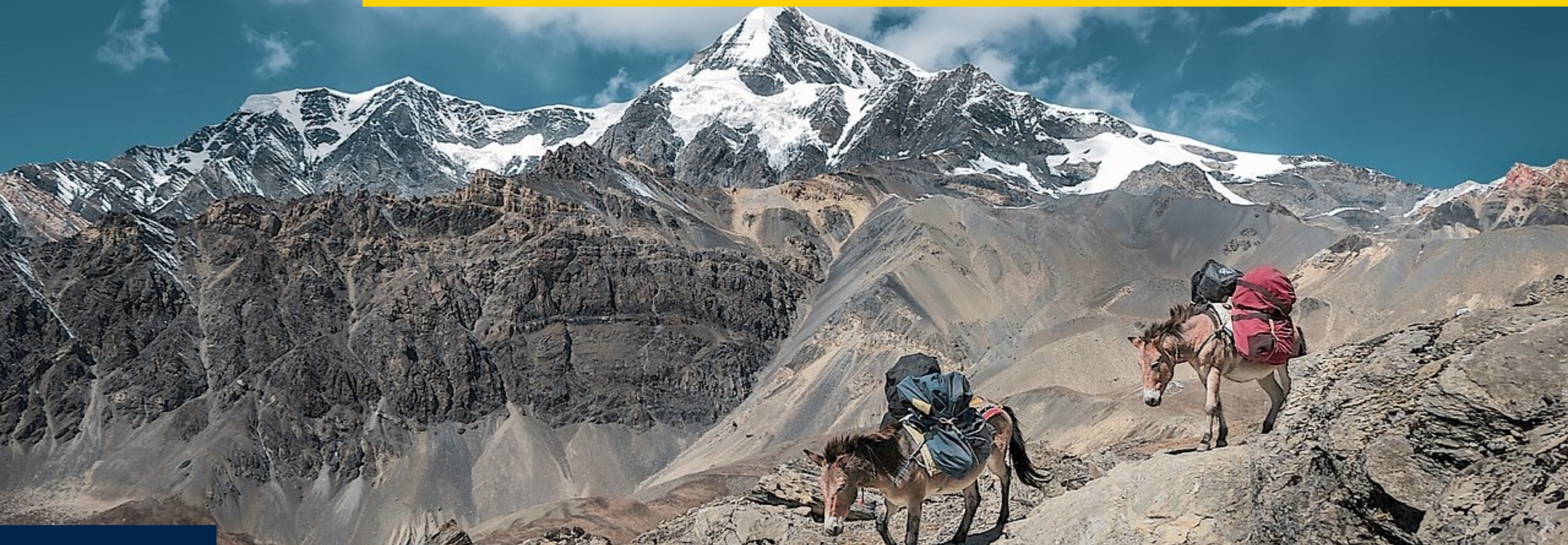
Loading...

What is bootloader

- Small program stored in FLASH
- Initialize the OSPI and OTFDEC
- Start code in OSPI



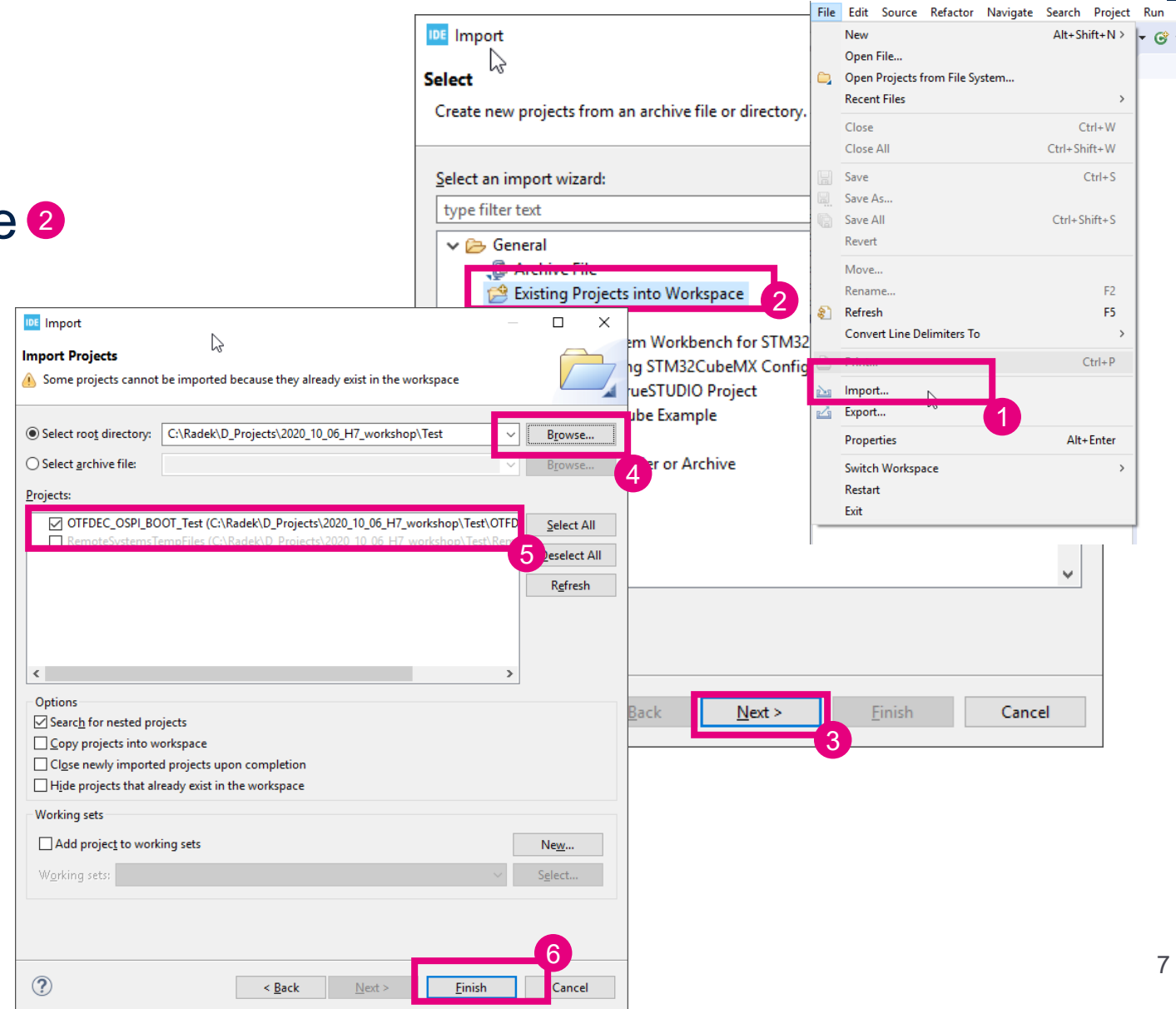
Creating bootloader



life.augmented

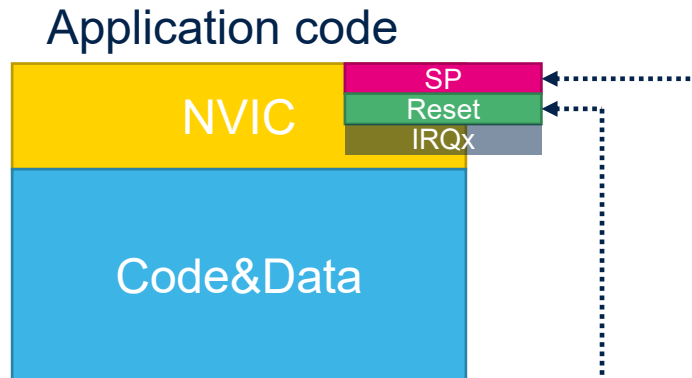
Import Bootloader to CubeIDE

- File>Import 1
- Select Import STM32CubeExample 2
- Next 3
- Browse
 - Only ok if project is in workspace folder 4
 - Select OTFDEC_OSPI_BOOT_Test 5 or HandsOn03
- Finish 6



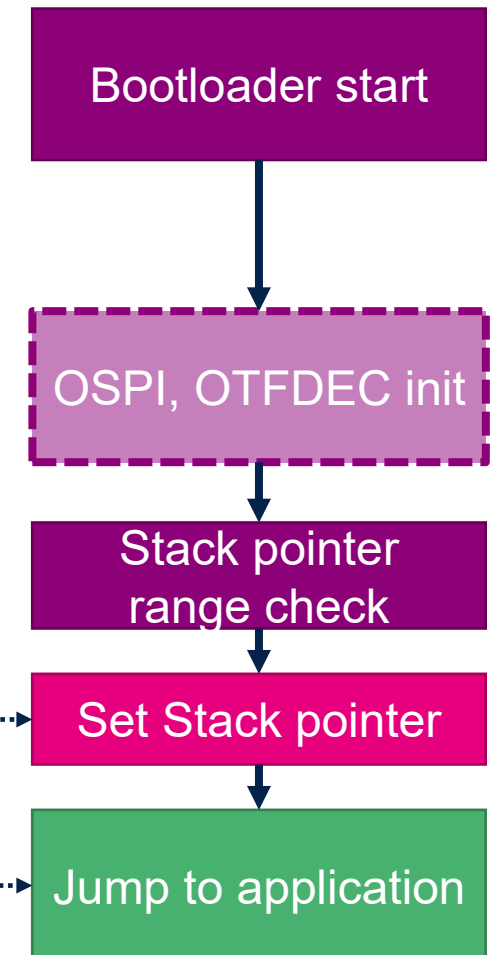
Use prepared project

- Project contains
 - CubeMX initialization
 - Jump to application



```
if (((*(__IO uint32_t *) IMAGE_DEFAULT_ADD) & 0x2FFC0000) >= 0x20000000)
&& (((*(__IO uint32_t *) IMAGE_DEFAULT_ADD) & 0x2FFC0000) <= 0x2FFC0000))
{
    /* Jump to user application */
    JumpAddress = (*(__IO uint32_t *) (IMAGE_DEFAULT_ADD + 4));
    /*address +4 is an reset vector*/
    JumpToApplication = (pFunction) JumpAddress;

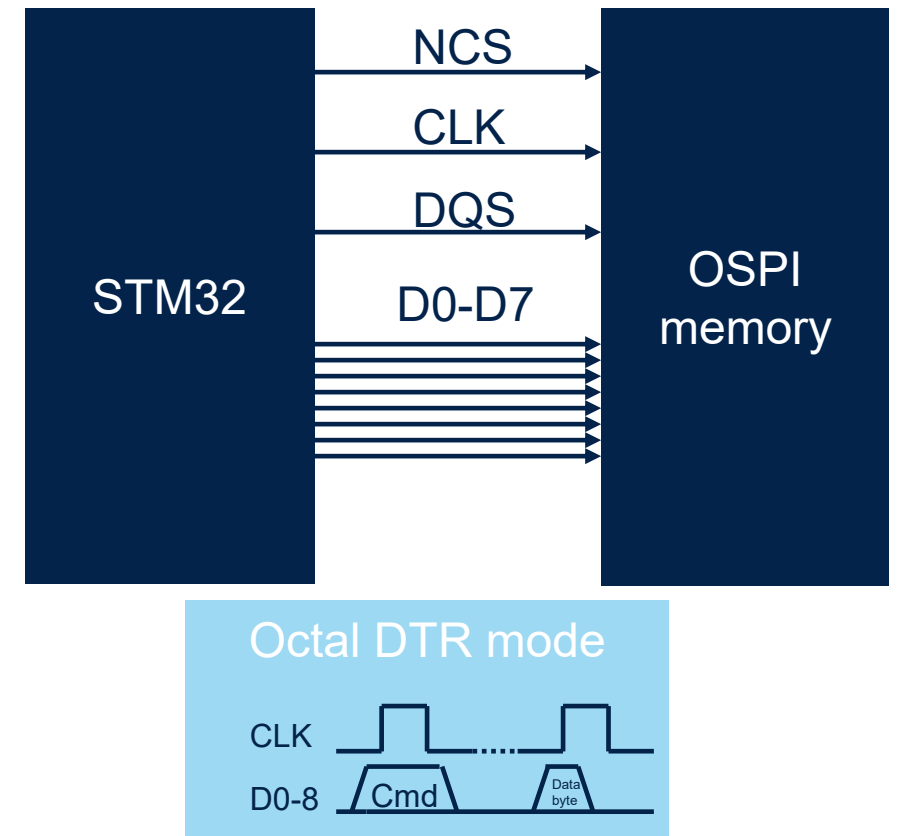
    /* Initialize user application's Stack Pointer */
    __set_MSP(*(__IO uint32_t *) IMAGE_DEFAULT_ADD);
    JumpToApplication();
}
```



OSPI memory STM32

OSPI memory

- Can use up to 8bit for data
- D0-D7 used for
 - Commands
 - Addressing
 - Data In/Out
- Possible use DTR mode
 - Putting data on rising and falling edge of clocks
- STM32 memory mapped mode
 - OSPI1 memory visible as STM32 memory
 - Address 0x90000000



Adding OSPI

- Open CubeMX
 - IOC project doubleclick
- Select OSPI1
 - OCTO SPI mode
 - Port1 CLK
 - Port1 NCS
 - Port1 DQS
 - Port1 IO[3:0]
 - Port1 IO[7:4]

The screenshot displays the STM32CubeMX interface for configuring the OCTOSPI1 peripheral. The 'Pinout & Configuration' tab is active, showing a list of peripherals. The 'Software Packs' section is expanded, showing the 'OCTOSPI1 Mode and Configuration' settings. The 'Configuration' section shows the 'Reset Configuration' and 'MDMA Settings' (NVIC Settings, GPIO Settings, Parameter Settings, User Constants). A circuit diagram at the bottom shows the connection of the OCTOSPI1 pins to the STM32L51245GXD100 microcontroller.

Pinout & Configuration

Categories: A-Z

System Core >
Analog >
Timers >
Connectivity >
Multimedia >
Security >
Computing >
Middleware >
Trace and Debug >
Power and Thermal >

Software Packs

OCTOSPI1 Mode and Configuration

Mode

Mode: Octo SPI
Clock: Port1 CLK
HyperBus(TM) 1.8V Inverted Clock: Disa...
Chip Select: Port1 NCS
Data Strobe: Port1 DQS (RWDS)
Data [3:0]: Port1 IO[3:0]
Data [7:4]: Port1 IO[7:4]

Configuration

Reset Configuration

MDMA Settings

NVIC Settings
GPIO Settings
Parameter Settings
User Constants

Configure the below parameters :

Circuit Diagram:

The diagram shows the connection of the OCTOSPI1 pins to the STM32L51245GXD100 microcontroller. The pins are connected to the following pins on the microcontroller:

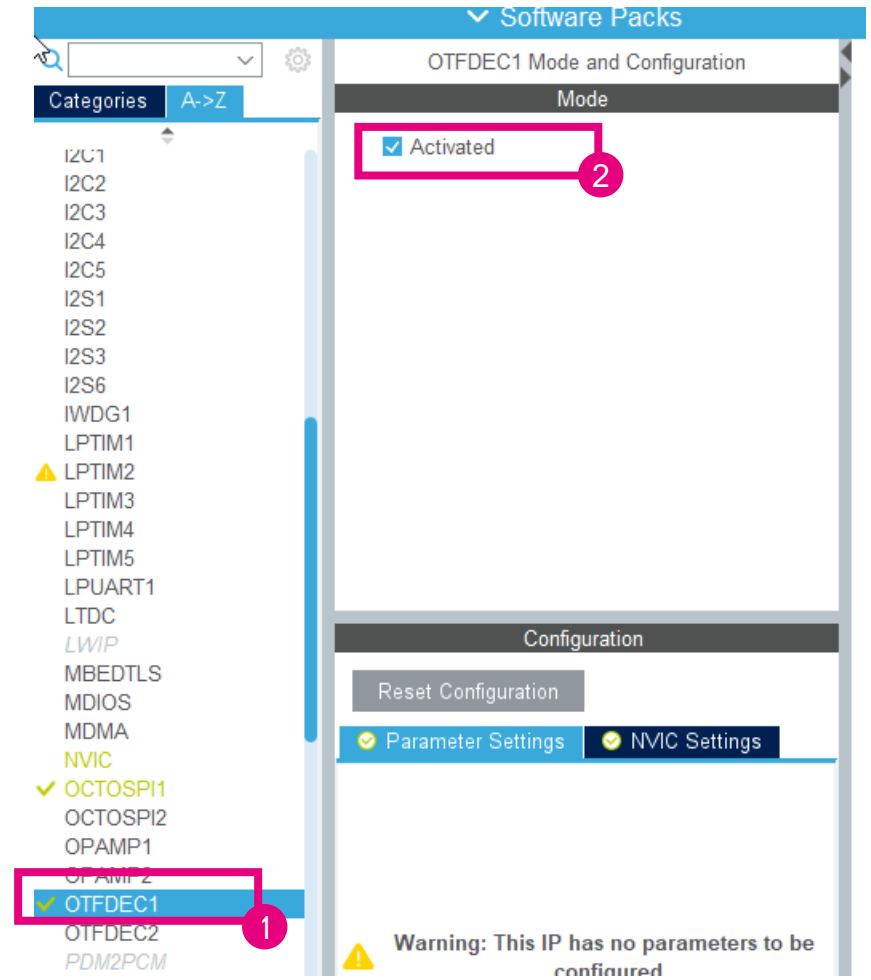
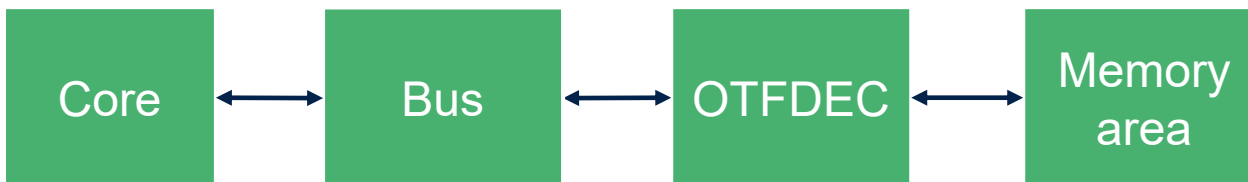
- OCSPI1_NCS: PG6
- OCSPI1_CLK: PF10
- OCSPI1_DQS: PB2
- OCSPI1_IO0: PD11
- OCSPI1_IO1: PD12
- OCSPI1_IO2: PE2
- OCSPI1_IO3: PD13
- OCSPI1_IO4: PD4
- OCSPI1_IO5: PD5
- OCSPI1_IO6: PG9
- OCSPI1_IO7: PD7
- NRST: SB26

The microcontroller is connected to a 3V3 supply and a 10K resistor (R81) to ground. The RESET pin is connected to a 3V3 supply and a 10K resistor (R102) to ground. The SB27 pin is marked as DNF (Not in File).

Adding OTFDEC

- Select OTFDEC1 1
- Activate 2
- Save
 - CTRL+S
 - CubeMX will regenerate the code

OTFDEC can cover any memory area



Run OSPI

- We will use prepared file ospi.c/h
- Files already in project
- We will only include ospi.h into main.c

```
/* USER CODE BEGIN Includes */  
#include "ospi.h"  
/* USER CODE END Includes */
```

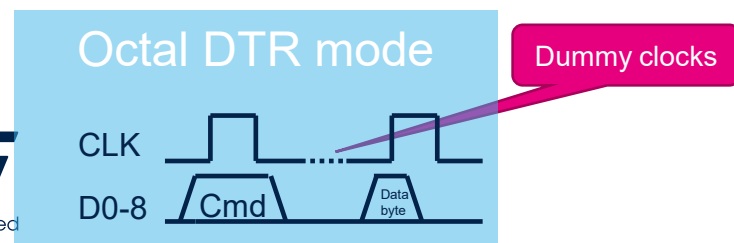
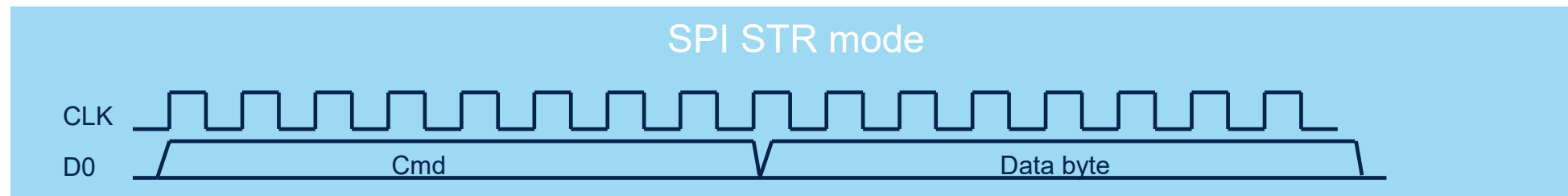
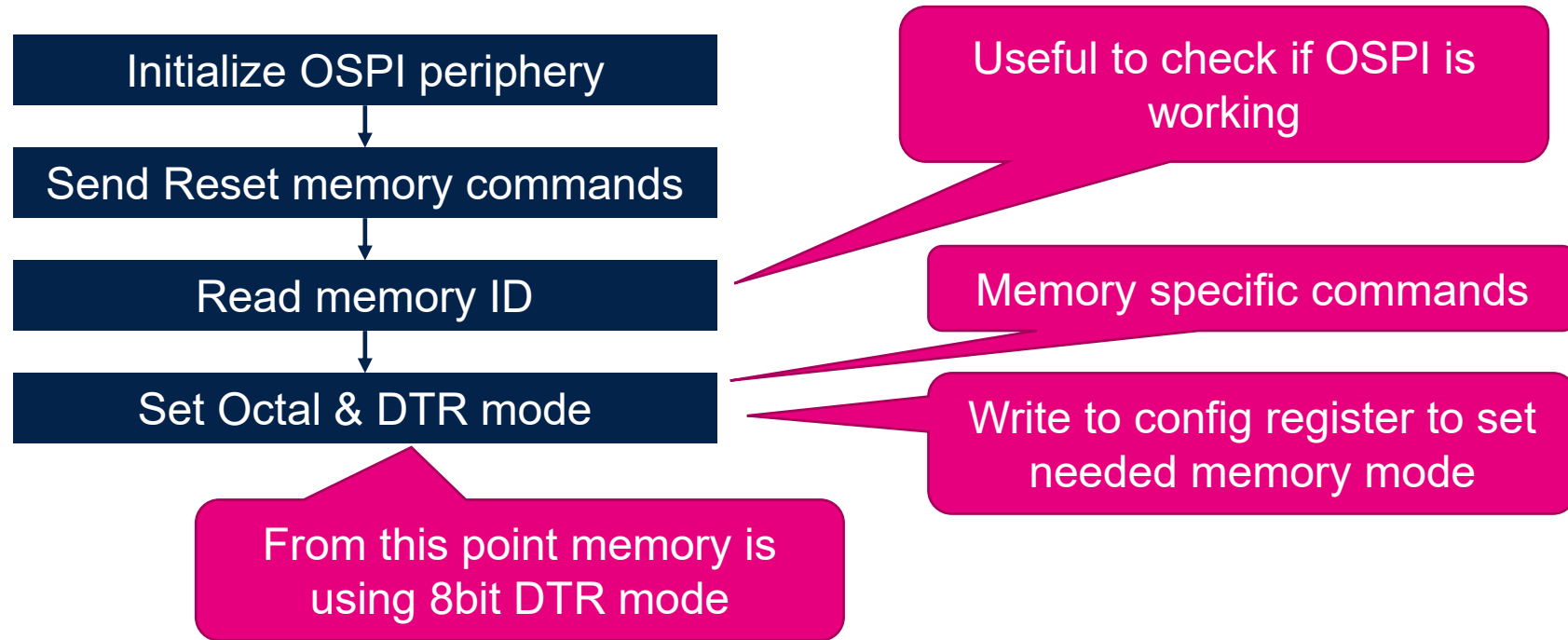
- And enable the OSPI by calling

```
/* USER CODE BEGIN 2 */  
OSPI_Init();  
OSPI_MemoryMap();
```

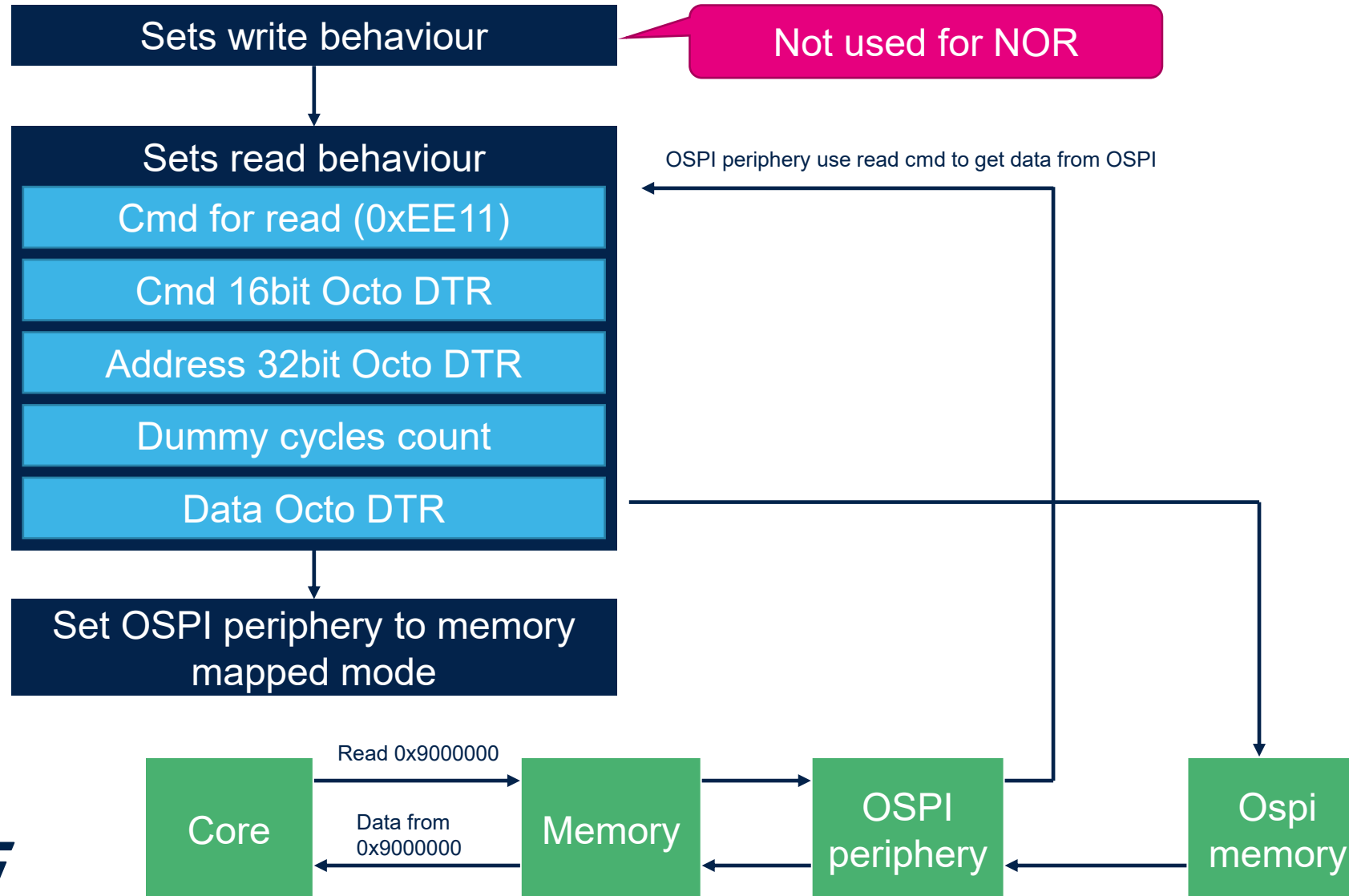
- Change start application start address to

```
/* USER CODE BEGIN PD */  
#define IMAGE_DEFAULT_ADD 0x90000000  
/* USER CODE END PD */
```

What OSPI Init do?



What OSPI_MemoryMap do?



We can load testing hex file

- Be sure no debugger is connected to board or STM32CubeProgrammer
- Run load_test_hex.bat to load the binary

```

C:\WINDOWS\system32\cmd.exe
-----
STM32CubeProgrammer v2.5.0-RC1
-----

ST-LINK SN : 002C003B3438510534313939
ST-LINK FW : V3J7M2
Board : STM32H735G-DK
Voltage : 3.28V
SWD freq : 24000 KHz
Connect mode: Normal
Reset mode : Software reset
Device ID : 0x483
Revision ID : Rev Z
Device name : STM32H72x/STM32H73x
Flash size : 1 MBytes
Device type : MCU
Device CPU : Cortex-M7

Memory Programming ...
Opening and parsing file: test_hex.hex
  File : test_hex.hex
  Size : 44 Bytes
  Address : 0x90000000

Erasing memory corresponding to segment 0:
Erasing external memory sector 0
Download in Progress:
100%

File download complete
Time elapsed during download operation: 00:00:00.354

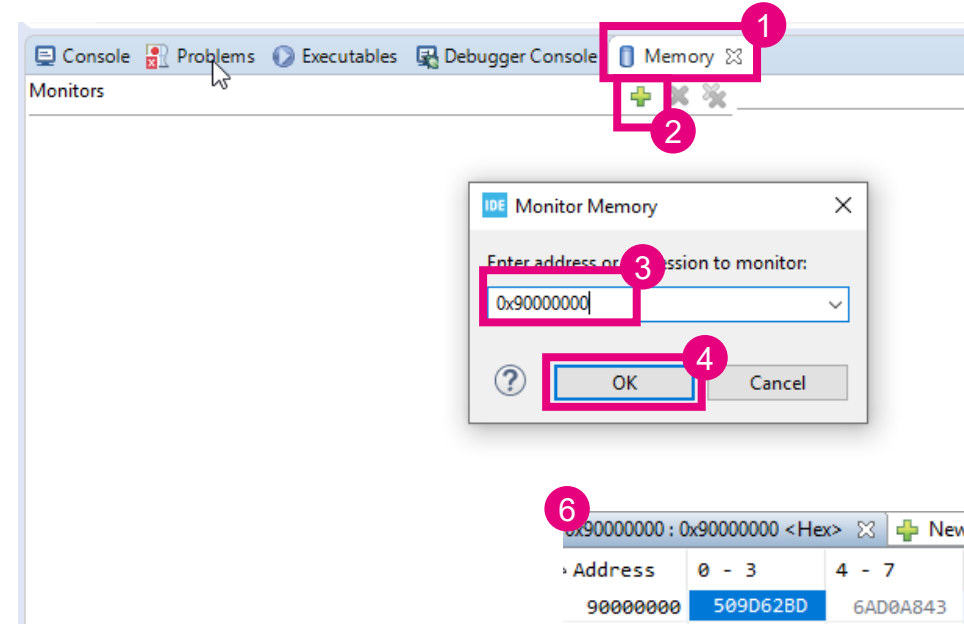
c:\Radek\D_Projects\2020_10_06_H7_workshop\Tools>ECHO "##### Flash OSPI end"
##### Flash OSPI end"

c:\Radek\D_Projects\2020_10_06_H7_workshop\Tools>pause
Press any key to continue . . .

```

We can test this in debug

- Run debug
- Run the code
- Open memory window at address 0x90000000 1-4
- Before OSPI initialized there will be no content in OSPI 5
- After initialization we can see some content 6



0x90000000 : 0x90000000 <Hex> + New Renderings...

Address	0 - 3	4 - 7	8 - B	C - F
90000000	00000000	00000000	00000000	00000000
90000010	00000000	00000000	00000000	00000000
90000020	00000000	00000000	00000000	00000000
90000030	00000000	00000000	00000000	00000000
90000040	00000000	00000000	00000000	00000000
90000050	00000000	00000000	00000000	00000000
90000060	00000000	00000000	00000000	00000000
90000070	00000000	00000000	00000000	00000000
90000080	00000000	00000000	00000000	00000000
90000090	00000000	00000000	00000000	00000000
900000A0	00000000	00000000	00000000	00000000
900000B0	00000000	00000000	00000000	00000000
900000C0	00000000	00000000	00000000	00000000
900000D0	00000000	00000000	00000000	00000000
900000E0	00000000	00000000	00000000	00000000
900000F0	00000000	00000000	00000000	00000000

0x90000000 : 0x90000000 <Hex> + New Renderings...

Address	0 - 3	4 - 7	8 - B	C - F
90000000	509D62BD	6AD0A843	A5CA91BF	E73EA966
90000010	6902375E	B1AFF02E	6EE2EBD6	364AB27C
90000020	9A170F42	8781E265	E72B1214	6E115D92
90000030	B543D11F	E48CC468	07A42AA1	E63A667D
90000040	57B09CD7	94DFC4E8	B6BD117A	84F95170
90000050	908D1A39	645F5BA3	EAA1AFED	CE2E06B9
90000060	4C3E8FDF	D05F1025	BE48E708	0FEEF0C3
90000070	2F317044	F79939C7	6813D831	AE5607D2
90000080	FD718282	55C07CF6	FDDFCC12	262A3465
90000090	9FC3C521	C7F1F899	006FE9B7	E02543AA
900000A0	0266EEE4	DDD3BA82	59F08349	0308A85B
900000B0	0023CAD0	3605BEA2	52965759	796E0DD0
900000C0	2D49BA12	FED58BA5	4807CBFA	41DEACAC
900000D0	83B98694	27D67A26	9CB8D860A	6882810A
900000E0	0256684B	1F6E4F7F	39320870	17294E84
900000F0	5B509545	C9C7427B	909E0287	6DAC9ECA

Modify application to run from OSPI

Change of linker and NVIC table location of HandsOn2

- Change NVIC table location (HandsOn02)

```
/* USER CODE BEGIN 1 */
```

```
SCB->VTOR= 0x90000000; /*STM32 will not search for interrupts at the beginning of the memory*/
```

```
/* USER CODE END 1 */
```

SCB->VTOR

NVIC table

Stack pointer

Reset addr

IRQ1

IRQ2

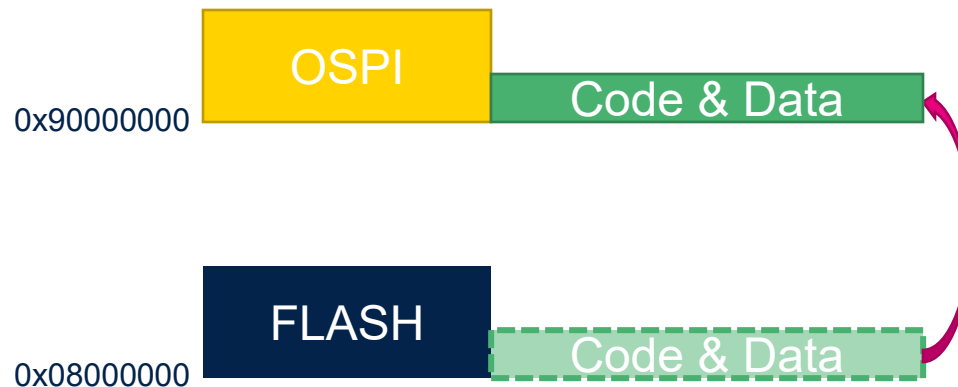
IRQ3

...

IRQx

- In linker STM32H735IGKX_FLASH.ld (HandsOn02)
change FLASH address from 0x08000000 to **0x90000000**

FLASH (rx) : ORIGIN = 0x90000000, LENGTH = 1024K



HandsOn2 OSPI clock source correction

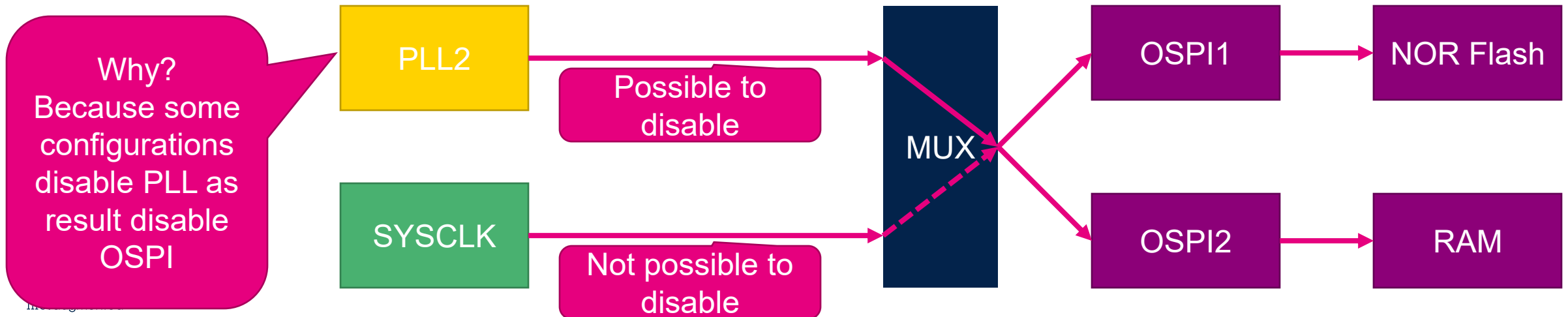
- In stm32h735g_discovery_ospi.c change (HandsOn02):

```
/* Fill config structure */
ospi_init.ClockPrescaler = 2U; /* OctoSPI clock = 200 MHz / ClockPrescaler = 100 MHz */
ospi_init.MemorySize      = (uint32_t)POSITION_VAL(S70KL1281_RAM_SIZE);
ospi_init.SampleShifting  = HAL_OSPI_SAMPLE_SHIFTING_NONE;

/* STM32 OSPI Clock configuration */
if (MX_OSPI_ClockConfig(&hospi_ram[Instance]) != HAL_OK)
{
    ret = BSP_ERROR_PERIPH_FAILURE;
}
```

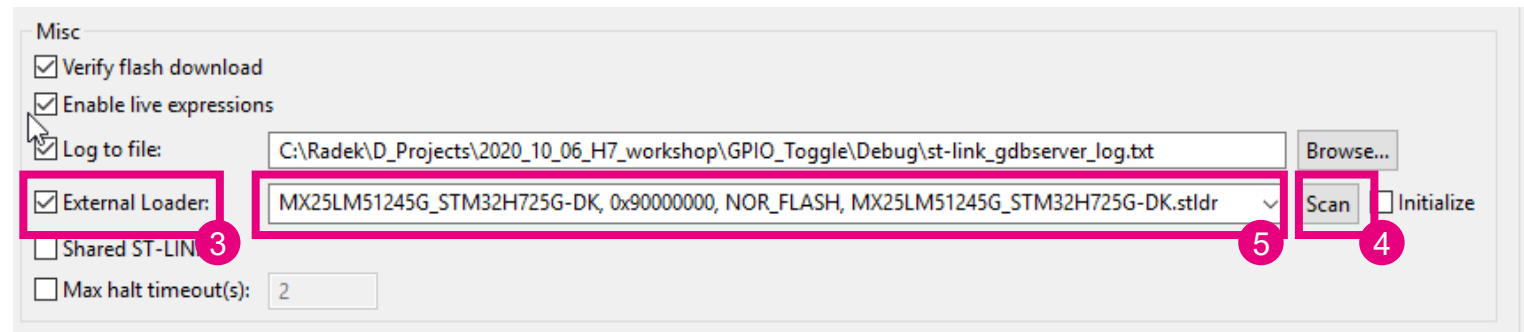
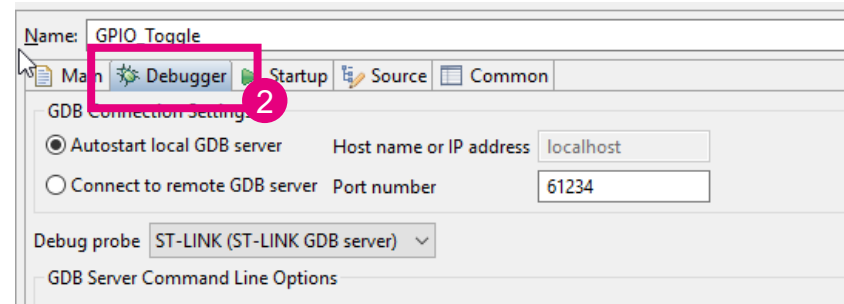
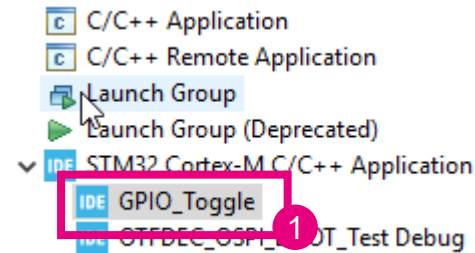
Annotations in the code:

- A pink arrow points from `ospi_init.ClockPrescaler = 3U;` to `ospi_init.ClockPrescaler = 2U;` in the first block.
- A pink dashed arrow points from `if (HAL_OK != HAL_OK)` to `if (MX_OSPI_ClockConfig(&hospi_ram[Instance]) != HAL_OK)` in the second block.



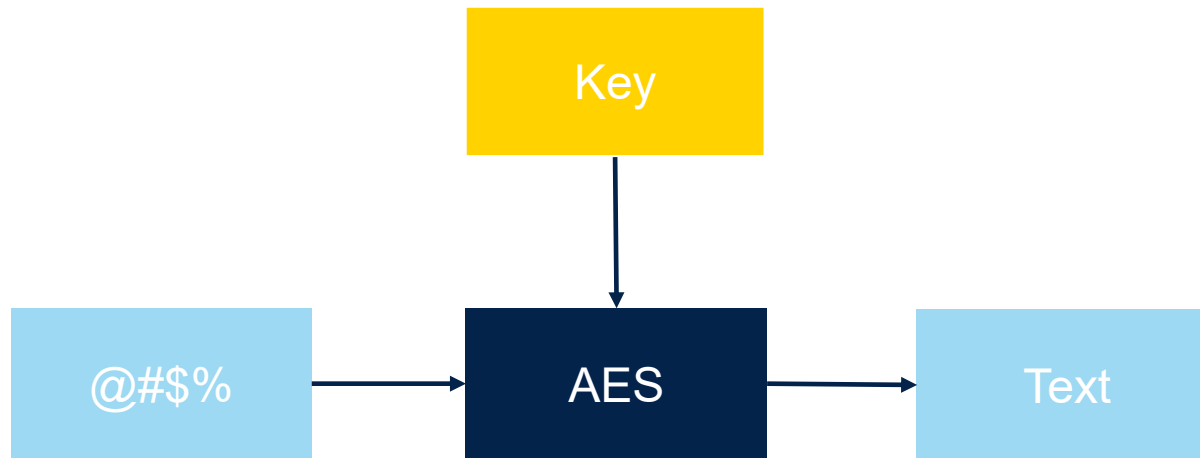
Load unencrypted binary

- Menu>Run>Debug configurations...
- Select current configuration ❶
- Tab>Debugger ❷
- check External Loader ❸
- Scan loaders ❹
- Select STM32H725/735-DK ❺

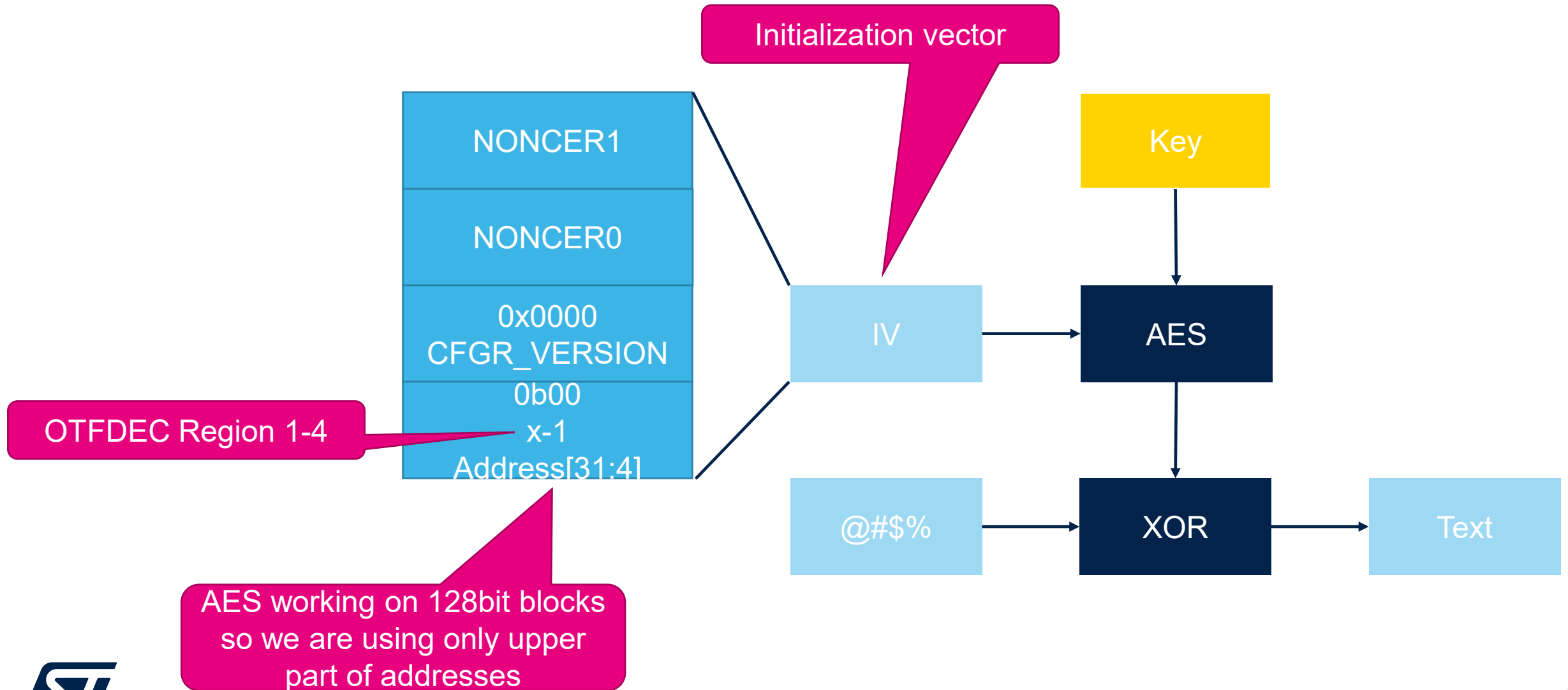


How OTFDEC(On The Fly Decryption) works

Decryption AES ECB



AES CTR



Adding decryption

Adding OTFDEC to our code

- Into main add OTFDEC_Init

```
/* USER CODE BEGIN 2 */  
OSPI_Init();  
OSPI_MemoryMap();  
OTFDEC_Init();
```

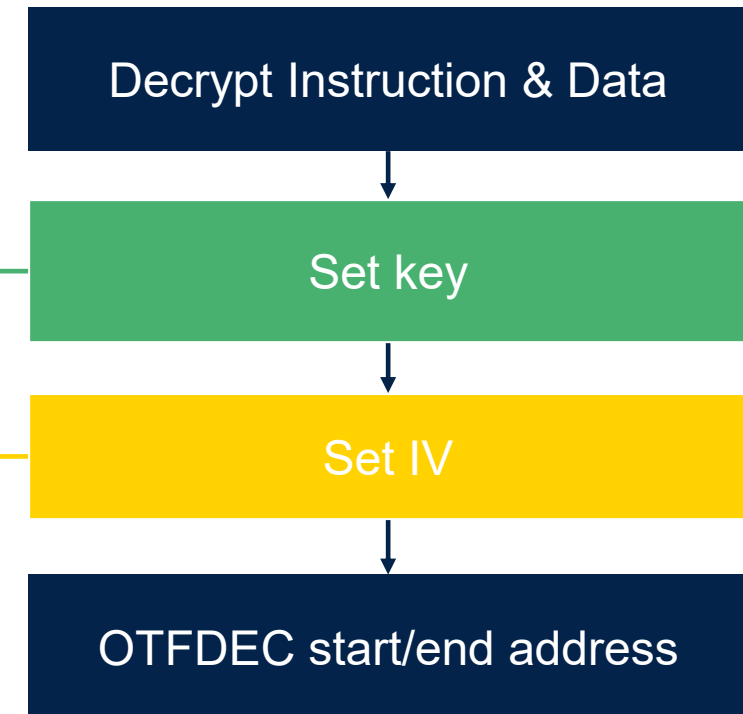
- OTFDEC_Init will

- We will set the region to decrypt instruction and data
- Set our KEY
- Init NONCE, VERSION and region start address

```
uint32_t Key[4] = { 0x12345678, 0x0, 0x0, 0x0 };
```

```
Config.Nonce[0]    = 0xA5A5A5A5;  
Config.Nonce[1]    = 0xC3C3C3C3;  
Config.StartAddress = START_ADDRESS_OTFDEC1_REGION1;  
Config.EndAddress   = END_ADDRESS_OTFDEC1_REGION1;  
Config.Version      = 0xABCD;
```

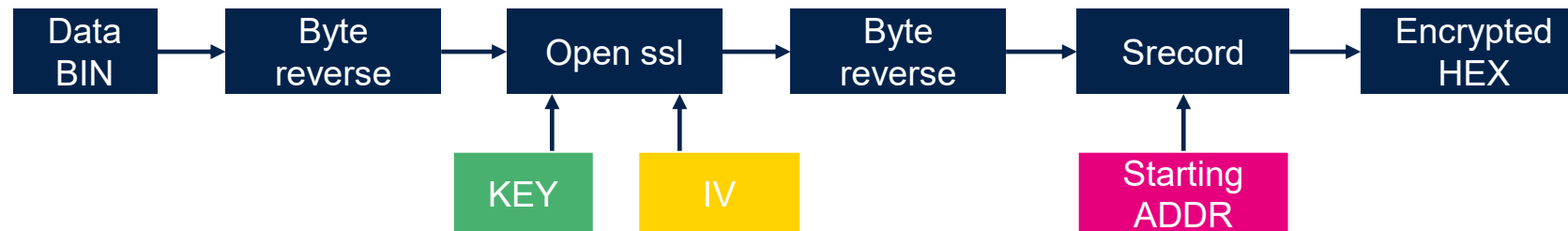
Area covered with
OTFDEC
From 0x90000000 to
0x9FFFFFFF



Creating encrypted binary/hex

How to create hex/binary

- We can use prepared batch file which is using
 - Open-ssl – to create encrypted data
 - Byte reverse utility – basically conversion between STM32 and PC byte order
 - Srecord – to convert BIN file HEX
 - STM32CubeProgrammer – to upload binary to STM32

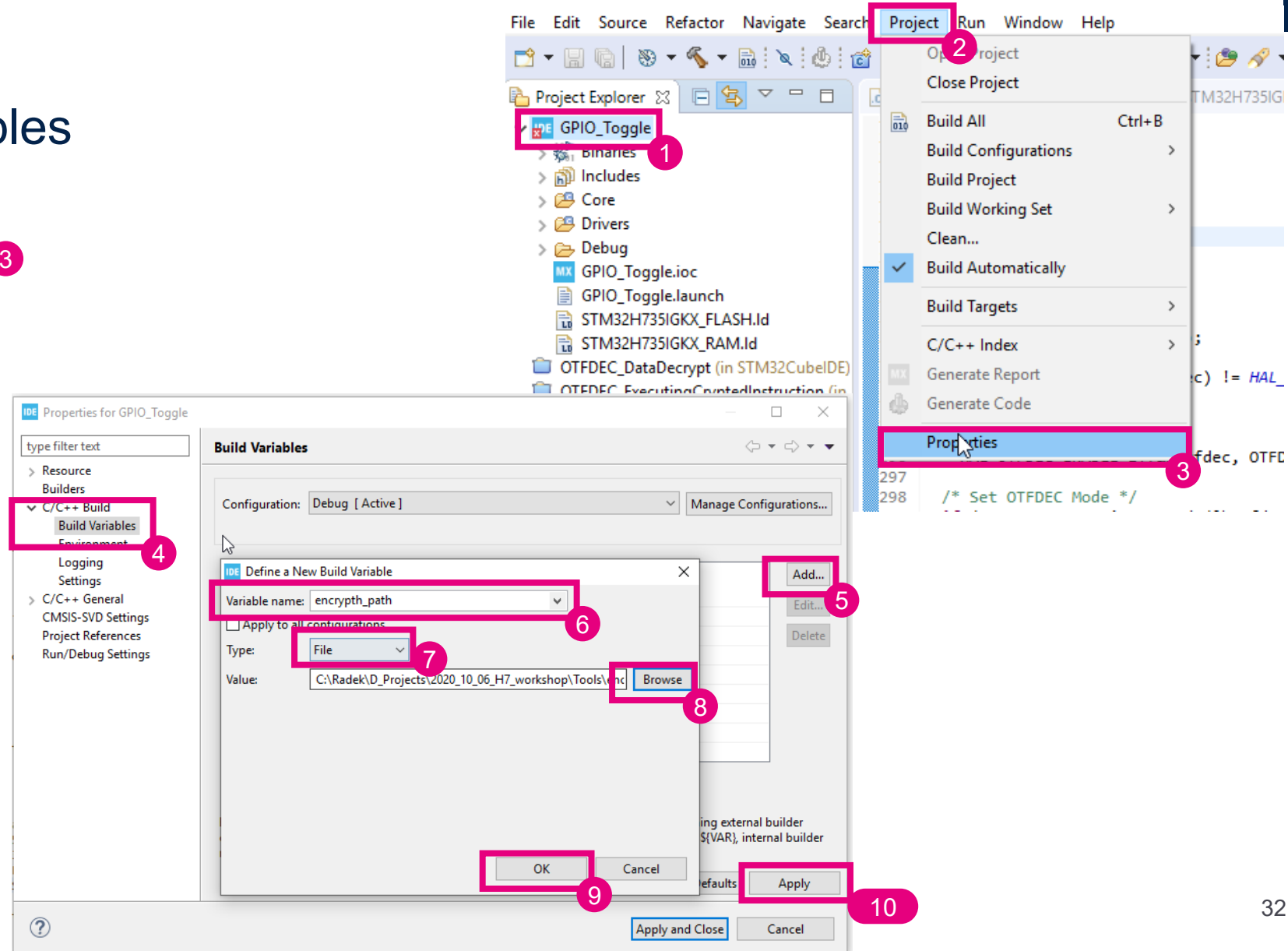


- Input parameters for batchfile



How to call batch file 1

- We can use eclipse variables
 - Mark project 1
 - Menu>Project>Properties 2 3
 - Build variables 4
 - Add.. 5
 - Name: encrypt_path 6
 - Type:File 7
 - Select path to 8
Tools/encrypt_bin.bat
 - OK 9
 - Apply 10



How to call batch file 2

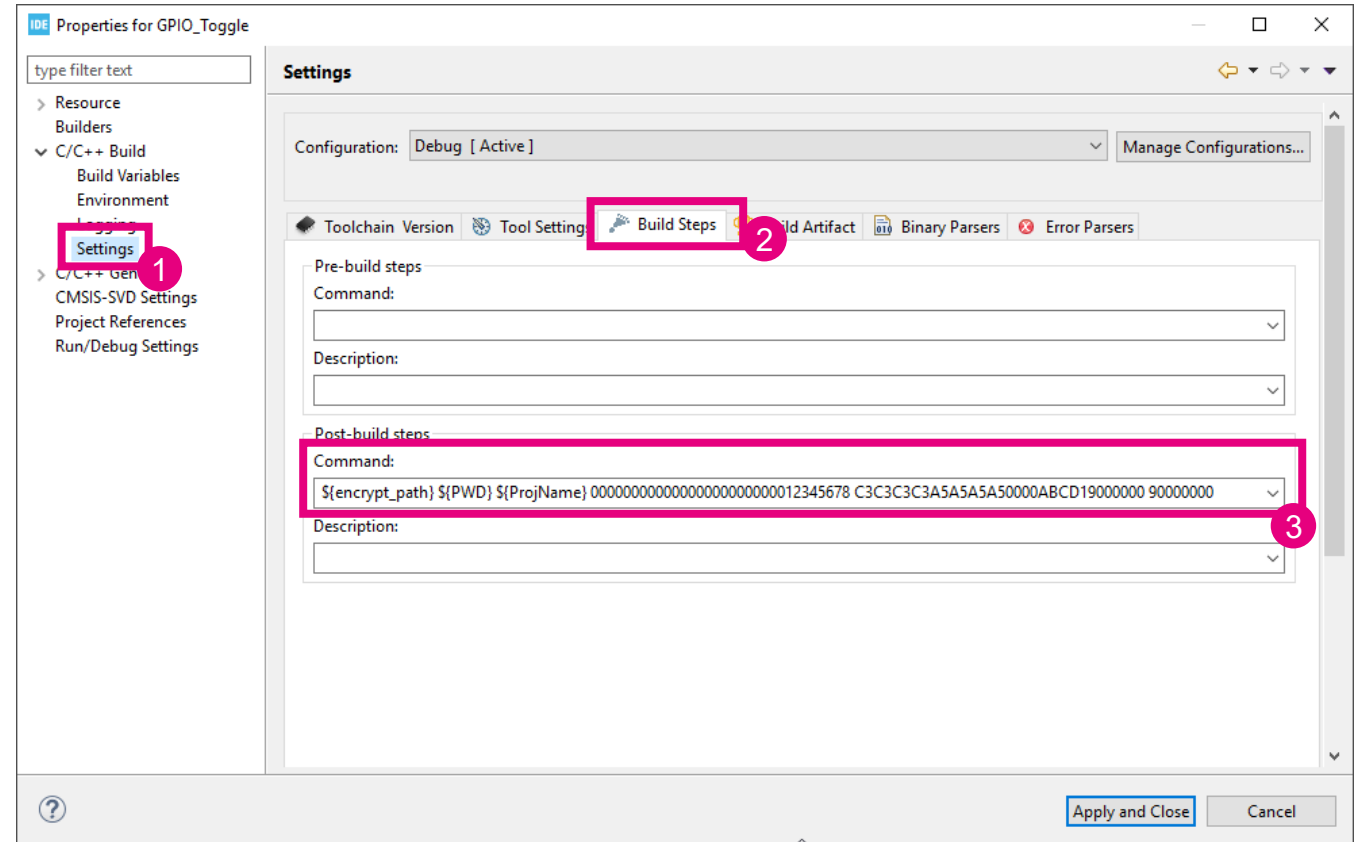
- Settings ①
- Build Steps ②
- Post build steps ③
- Insert

`${encrypt_path} ${PWD} ${ProjName}`

encrypt_bin.bat

Input file path

Input file name

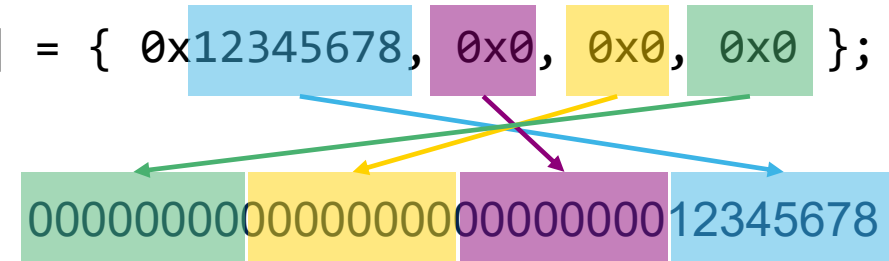


How to call batch file 3

- Filling the Key

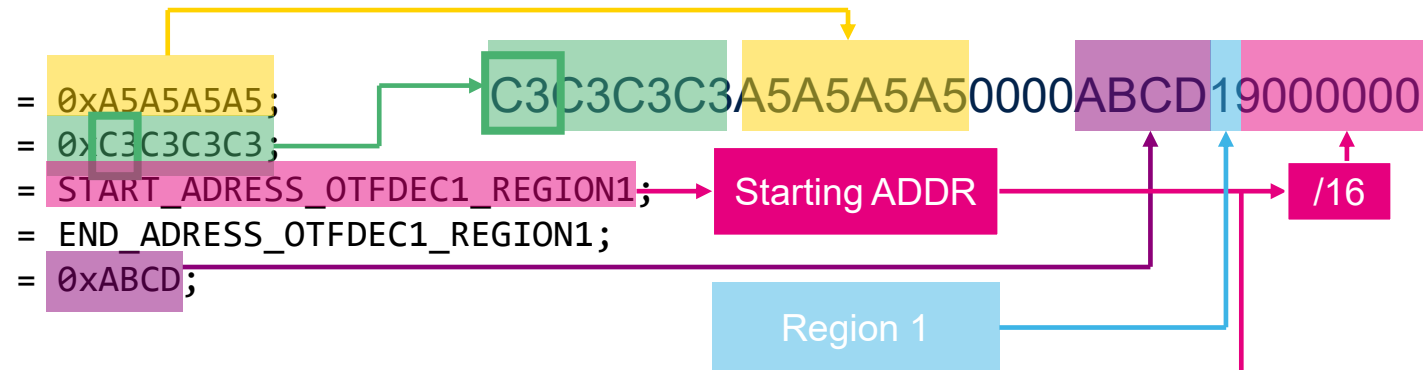
- Key representation on STM32
- Key representation for open-ssl

```
uint32_t Key[4] = { 0x12345678, 0x0, 0x0, 0x0 };
```



- Filling IV

```
Config.Nonce[0] = 0xA5A5A5A5;
Config.Nonce[1] = 0xC3C3C3C3;
Config.StartAddress = START_ADDRESS;
Config.EndAddress = END_ADDRESS;
Config.Version = 0xABCD;
```



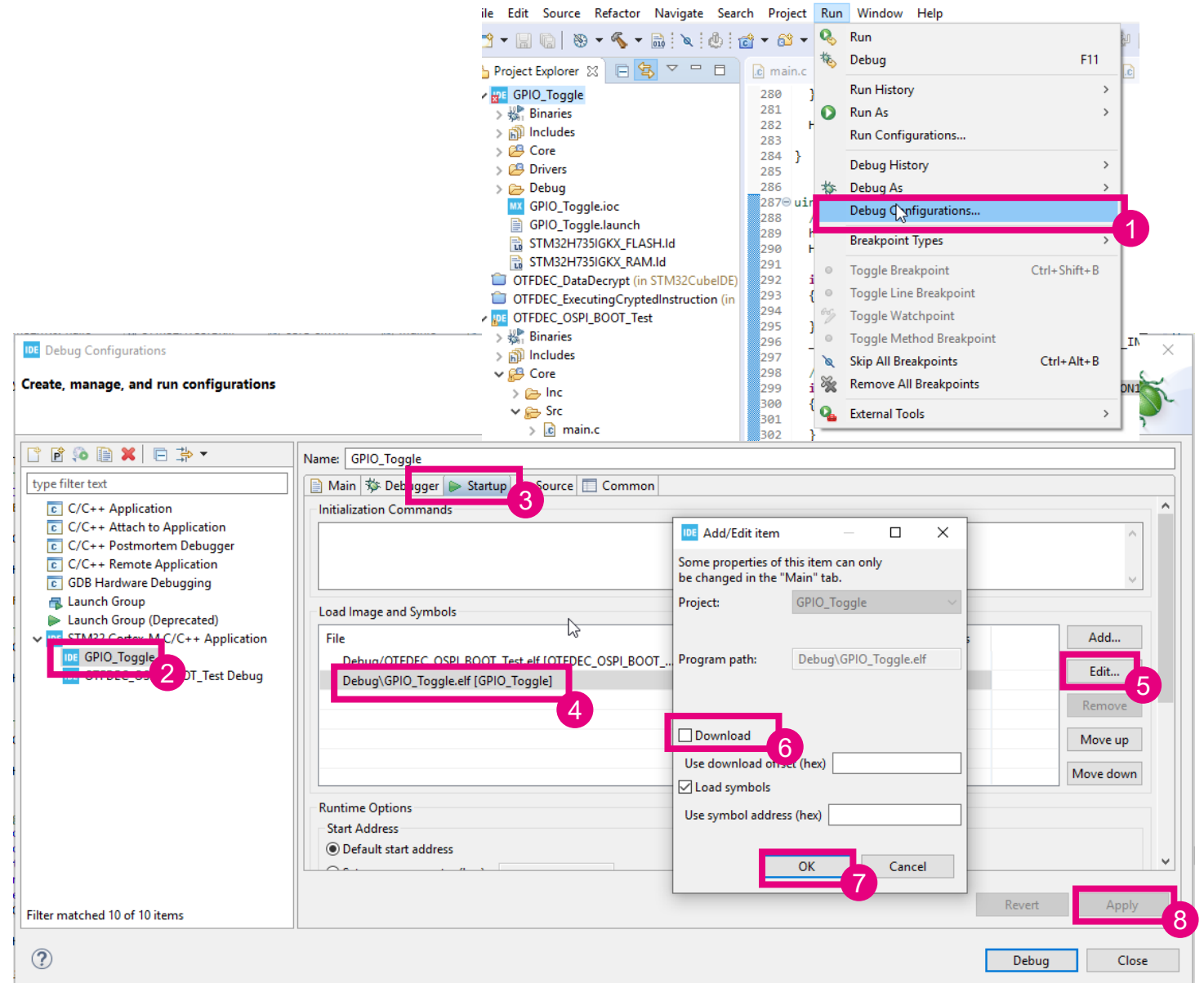
- Filling address

- OSPI memory position 0x90000000 => 90000000

[illegible]

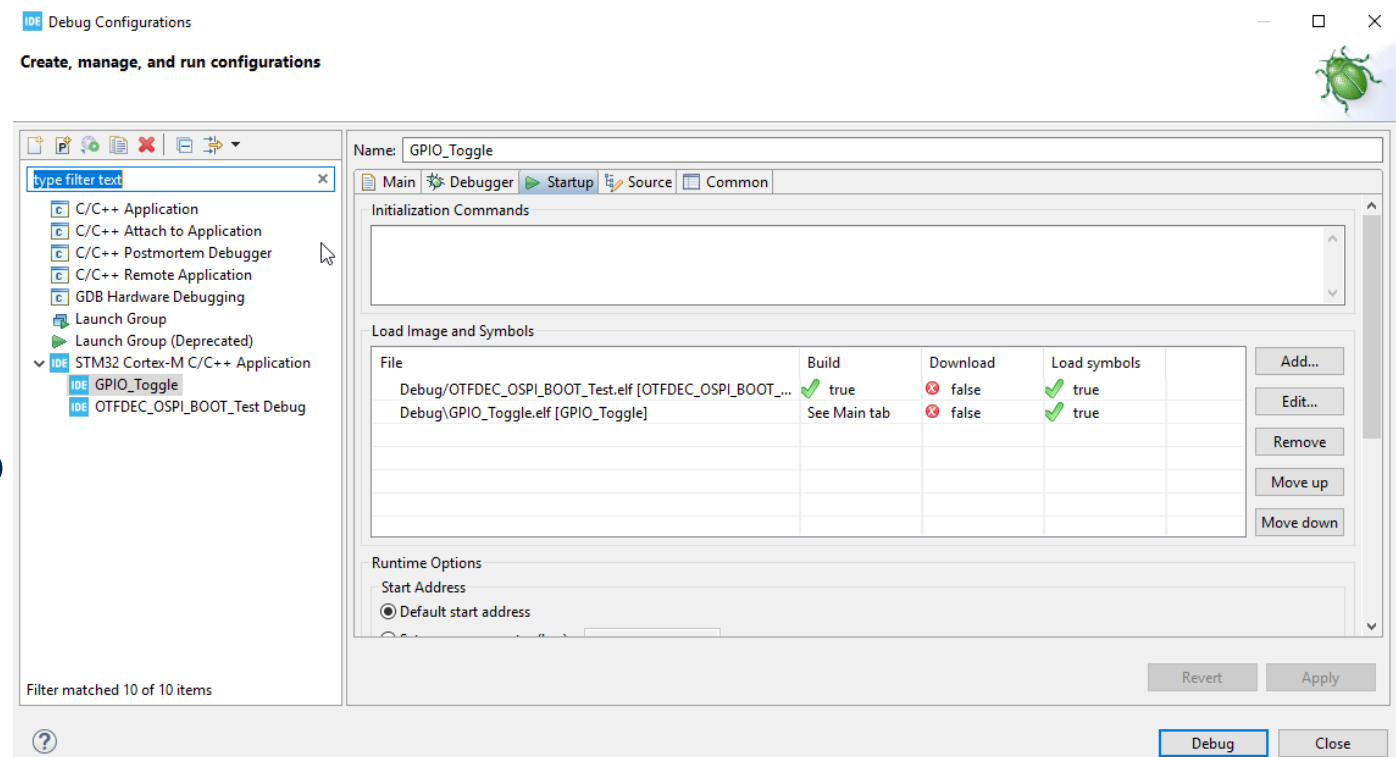
Before debugging

- Run>Debug Configuration ①
- Select project(HandsOn02) ②
- Startup ③
- Select project elf file ④
- Edit ⑤
- Uncheck Download ⑥
 - Our bat file will load our code
- OK ⑦
- Apply ⑧



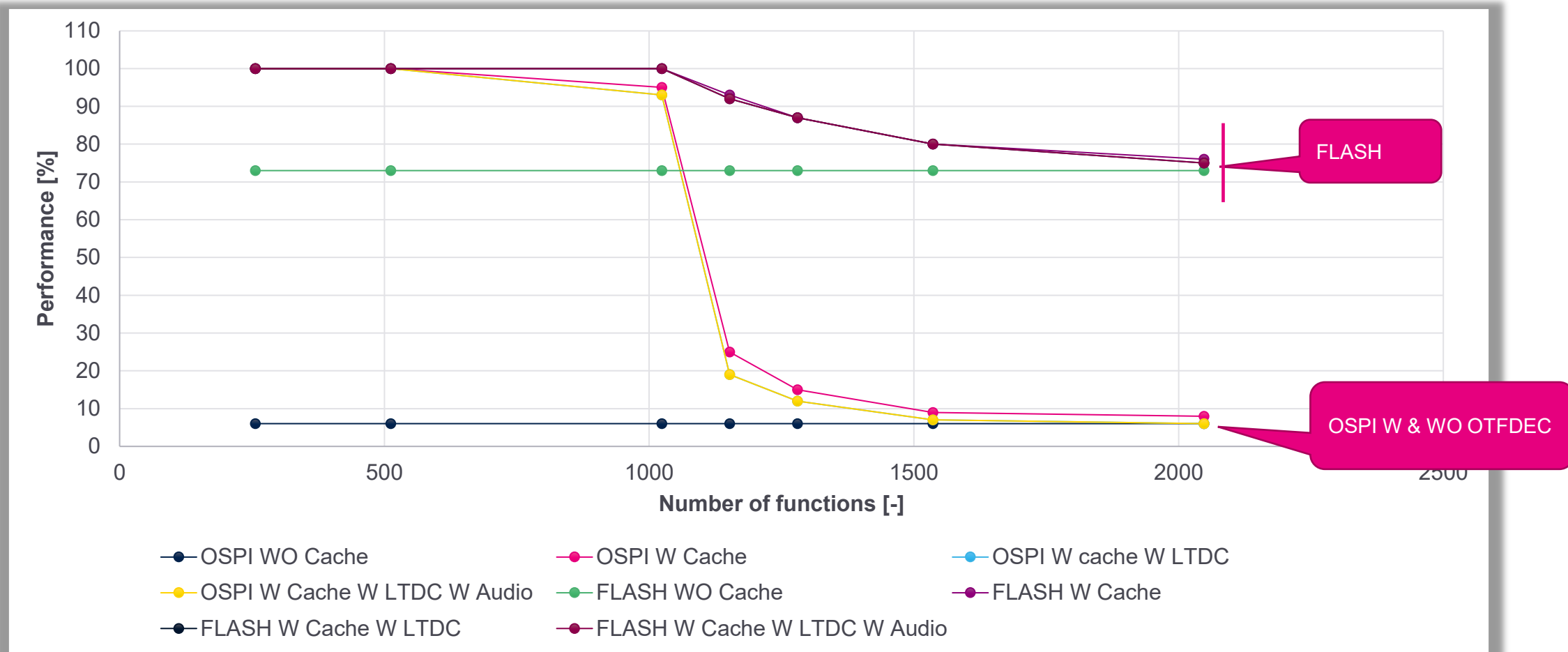
Use out project

- To build and download code
 - Use Build option
 - CTRL+B
- Debug
 - Only for debug
 - No code lading
- Debug also loader
 - Possible to add loader elf file in Startup



OSPI execution results

Results



Graph data

Number of functions [-]	Performance [%]											
	OSPI OTFDEC				OSPI				FLASH			
	OSPI OTFDEC WO Cache	OSPI OTFDEC W Cache	OSPI OTFDEC W Cache W LTDC	OSPI OTFDEC W Cache W LTDC W Audio	OSPI WO Cache	OSPI W Cache	OSPI W Cache W LTDC	OSPI W Cache W LTDC W Audio	FLASH WO Cache	FLASH W Cache	FLASH W Cache W LTDC	FLASH W Cache W LTDC W Audio
256	6	100	100	100	6	100	100	100	73	100	100	100
512	6	100	100	100	6	100	100	100	73	100	100	100
1024	6	95	93	93	6	95	93	93	73	100	100	100
1152	6	25	19	19	6	25	19	19	73	93	92	92
1280	6	15	12	12	6	15	12	12	73	87	87	87
1536	6	9	7	7	6	9	7	7	73	80	80	80
2048	6	8	6	6	6	8	6	6	73	76	75	75

LTDC bandwidth => $60\text{Hz} * 472\text{H} * 272\text{W} * 3\text{Bpp} = 23\text{MB/s}$

OSPI bandwidth => $91.67\text{MHz} * 2\text{DTR}$

Conclusion

- Code execution from internal flash is constant with cache disabled
- When use Cache for data loading
 - OSPI have almost similar performance as internal flash up to some extent (when code is fully in cache)
 - Until cache is depleted then performance is dropping
 - Complex applications may create cache misses and decrease performance from external flash
- Is better to allocate Cache to OSPI then to Flash if possible
- OTFDEC have no performance impact on OSPI
- Static non-volatile memory transfers have influence to CPU performance
 - When using OCTOSPI memory
 - Thanks to OCTOSPI memory, influence is not highly critical, QSPI would be problematic more

Thank you

© STMicroelectronics - All rights reserved.

ST logo is a trademark or a registered trademark of STMicroelectronics International NV or its affiliates in the EU and/or other countries.

For additional information about ST trademarks, please refer to www.st.com/trademarks.

All other product or service names are the property of their respective owners.



life.augmented